Decision Orchestration: Step-by-Step Annotation of Complex Ontologies that Improves Itself from a Handful of Gold Sentences

Anonymous ACL submission

Abstract

Annotating a complex ontology needs both an understanding of the structure it describes and the scheme's codebook. Early in a project, when gold data is scarce, large language models can bootstrap the annotation, but a guideline of hundreds of pages is too long to follow in context, and when a model writes the whole annotation at once, its errors cannot be traced to the rule it misapplied. We propose *decision orchestration*: the guideline is compiled into a procedure of small steps in which code cuts, enumerates and assembles, and models are consulted only at decision nodes, either writing a small schema-checked fragment or answering closed questions with probabilities that rules read against thresholds. Every output unit comes from a logged step, so every error has an address, and a reflector LLM uses these addresses to edit the procedure from five gold sentences at a time. On samples of Uniform Meaning Representation (the 2026 shared task) and PARSEME verbal multiword expressions, in Chinese and English, seed procedures written from the guidelines alone, with no examples and decided by small LLMs or by decision models that never write text, match or beat a frontier model given the whole guideline on three of the four task–language pairs (Chinese UMR: 35.0 against 23.9; Chinese PARSEME: 62.5 against 50.0), though not on English PARSEME (62.3 against 75.6). Fifty gold sentences of reflection improve twelve of fourteen procedures, by up to 31 points; the best Chinese PARSEME procedure reaches 72.5, the best UMR procedure (Chinese, 37.6; 38.2 with a cascade) is decided by a model that costs under 30 cents per hundred sentences, and on English PARSEME every reflected procedure exceeds the best shared-task system on the same sentences. The procedures stay readable and diagnosable throughout, which makes them a practical first step for new annotation projects.

1 Introduction

Annotating a complex ontology takes two things: an understanding of the structure the scheme describes, and the scheme's own codebook, the conventions that make one annotation right and a plausible alternative wrong (Kim et al., 2026). Early in a project, a new scheme, language or domain has a guideline but little gold. With enough gold, supervised models learn both (Bevilacqua et al., 2021; Bai et al., 2022); with a handful of sentences they learn neither. Large language models bring the first ingredient for free: they understand sentences, events and referents well enough to bootstrap an annotation. What remains is to give them the codebook.

For simple labeling, the guideline fits into the prompt (Gilardi et al., 2023; Tan et al., 2024; Kim et al., 2026). For complex schemes it does not. The UMR guidelines run to more than 300 pages (Van Gysel et al., 2021); put into one context, the rules that matter for a given sentence sit in the middle of a long document, where models use them least (Liu et al., 2024), and an annotation must satisfy dozens of them at once, where following degrades with each added instruction (Harada et al., 2025). A second problem is just as practical. At the start of a project, people and models correct each other's errors, and correcting an error needs a diagnosis: which rule was misapplied, and where. When the whole guideline sits in one prompt and the model writes the whole annotation, an error cannot be traced to a rule, and fixing it means rewording a prompt and hoping.

We propose **decision orchestration** (Figure 1): compile the guideline into a procedure of small steps and run them in order. Code (rule nodes) cuts the input, enumerates candidates and assembles the output; models are consulted only at decision nodes, each with the part of the guideline it needs. A decision node is either a *fragment* de-

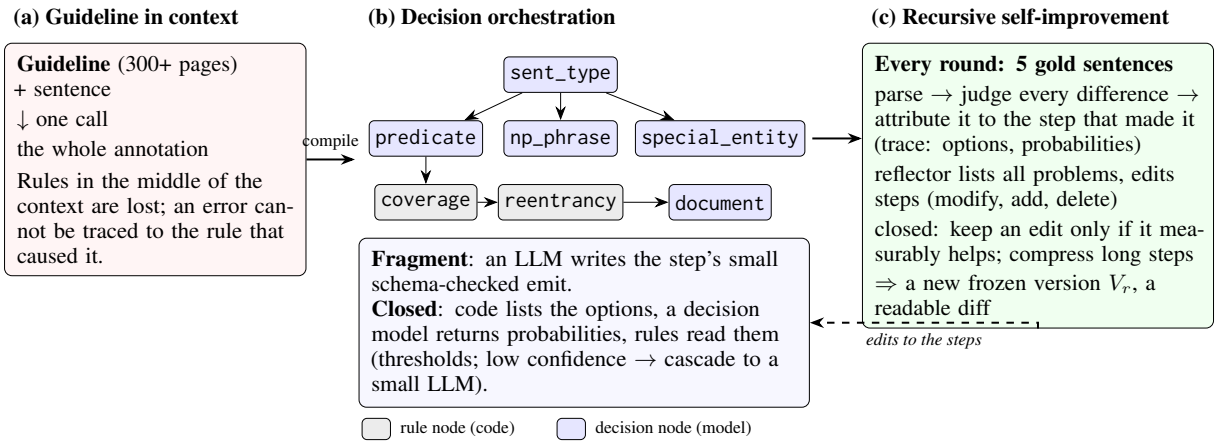


Figure 1: (a) The usual way to give an LLM a codebook: the whole guideline in context. (b) Decision orchestration compiles the guideline into a procedure of small steps, rule nodes and decision nodes, dispatched by a registry; a decision node is either a fragment decision by an LLM or closed questions answered by a decision model. (c) With a handful of gold sentences per round, a reflector edits the steps that made the errors.

cision, in which an LLM writes a small schema-checked piece of the annotation, or a set of *closed* questions whose options code has listed, answered with probabilities by a decision model and read by rules with thresholds. Because every output unit comes from a logged step, every error has an address. A reflector LLM uses these addresses to improve the procedure itself from five gold sentences at a time: it lists every problem it finds, edits the responsible steps, and in closed mode keeps only edits that measurably help; we call this recursive self-improvement (RSI) of the procedure, as the model weights never change.

To our knowledge, this is the first work to annotate complex schemes step by step along their guidelines, and we show that it pays twice: (1) errors are diagnosable and can be fixed where they arise, and (2) with very little data, RSI makes the procedure much better than the guideline it started from and than a frontier model given the whole guideline. We use small LLMs (Claude Haiku 5.5, and GPT-6-luna on PARSEME) and, beyond them, fast decision models that output only probabilities (Jev, the OpenAI Decisions API, Microsoft-Decision-1), with a mid-size reflector (Claude Sonnet 5.5). We test on samples of two complex benchmarks, each in Chinese and English: the test sets of the 2026 UMR parsing shared task (Zeman and Štěpánek, 2026) and PARSEME verbal multiword expressions (Ramisch et al., 2018, 2020), starting from seeds written from the guidelines alone, with no examples. Our findings:

- **Steps beat the guideline in context.** Without any gold, the best seed procedure matches or beats GPT-6.1-sol given the whole guideline on three of four task–language pairs; on Chinese UMR every closed seed is 8–11 points ahead (Jev 33.9 against 23.9 juːmætʃ F1). On English PARSEME, a scheme close to what frontier models already know, the frontier model stays ahead (§5).

- **A handful of gold goes a long way.** Ten rounds of reflection on five gold sentences each improve twelve of fourteen procedures, by up to 31 points (Jev on English PARSEME, 22.5 → 53.7). The best UMR procedure (Chinese, 37.6) is decided by Jev at under 30 cents per hundred sentences, and on English PARSEME every reflected procedure is above the best shared-task system rescored on the same sentences.

- **Errors have addresses.** The step-level judge shows where the two kinds of decision fail: closed procedures lose where the code offers no right option (e.g., an English lemmatizer that leaves *stolen* unreduced), fragment procedures where structure is left to the model; measured edits keep closed reflection from drifting, unmeasured fragment edits do not (§5.1, §6).

These results suggest that LLMs can learn an annotation scheme from its instructions and a few dozen gold sentences, keep improving as new gold arrives, and that the models doing the annotating need not be frontier models: even simple decision models can serve.

2 Related Work

LLMs as annotators. LLMs can replace or assist crowd workers on many labeling tasks (Wang et al., 2021; Gilardi et al., 2023; Törnberg, 2023; He et al., 2024; Ziems et al., 2024; Tan et al., 2024), but their quality varies by task and must be validated against human labels (Pangakis et al., 2023). Human–LLM collaboration allocates uncertain items to people or asks people to verify labels (Li et al., 2023; Wang et al., 2024). When a benchmark has conventions of its own, zero-shot labels drift from them; putting the guideline into the prompt and refining it in rounds of moderation brings LLM annotation closer to the gold (Kim et al., 2026), and guidelines can also be learned from errors (Pang et al., 2023) or written as code that a model is trained to follow (Sainz et al., 2024; Wang et al., 2023). All of these keep the guideline as one block of context for one model call. For complex ontologies such as UMR, whose guidelines run to hundreds of pages (Van Gysel et al., 2021; Bonn et al., 2025), that block exceeds what a model reliably uses: performance falls for information in the middle of long contexts (Liu et al., 2024) and drops roughly exponentially with the number of instructions to be followed at once (Harada et al., 2025). Expert-level structured annotation by prompting remains weak (Ettinger et al., 2023; Sun et al., 2024), and supervised parsers need large corpora (Bevilacqua et al., 2021; Bai et al., 2022; Chun and Xue, 2024; Markle et al., 2025). We instead compile the guideline into a procedure of small steps, so that each model call sees only the part of the guideline it needs, and an error can be traced to the step that made it.

Recursive self-improvement and rule-based reflection. The idea that a system can improve the procedure it runs goes back to Good (1966) and to self-referential learners (Schmidhuber, 2007). Before neural models, the most successful annotation learners in NLP learned readable rules from a little gold: transformation-based learning starts from a simple annotator and greedily adds the rewrite rule that most reduces errors against gold (Brill, 1992, 1995; Ramshaw and Marcus, 1995; Ngai and Florian, 2001); rule induction (Cohen, 1995) and heuristic learning (Mitchell et al., 1983; Lenat, 1983) similarly propose and refine heuristics and keep those that work on examples; data programming combines labeling functions written by people (Ratner et al., 2017). With LLMs, prompts, programs and agent scaffolds are optimized from feedback (Zhou et al., 2023; Pryzant et al., 2023; Yang et al., 2024; Khattab et al., 2024; Opsahl-Ong et al., 2024; Yuksekgonul et al., 2025), by reflecting on execution traces (Agrawal et al., 2026; Shinn et al., 2023; Madaan et al., 2023), by growing playbooks or rule sets from experience (Zhang et al., 2026b; Zhao et al., 2024; Zhu et al., 2023), or by letting systems rewrite their own code under empirical validation (Zelikman et al., 2024; Hu et al., 2025; Zhang et al., 2025, 2026a; Novikov et al., 2025). Self-feedback without external signal is unreliable (Huang et al., 2024). Our reflection is closest to transformation-based learning: gold annotations, not a model's opinion, decide which edits stay, the learned object is a readable procedure rather than weights, and each round is a small set of rewrites of rules, options and instructions. Unlike TBL's single rule template, the reflector may edit any part of the procedure, and unlike most prompt optimizers, it attributes each error to the step that made it.

3 Decision Orchestration

The tasks. Figure 2 shows what the two schemes ask for. A UMR annotation of a sentence is a graph whose concepts must follow a lexicon (PropBank) and a large inventory of abstract concepts, roles and attributes, aligned to the tokens, and a document-level layer that links each event to a temporal anchor and a conceiver and each mention to its antecedent (Van Gysel et al., 2021; Bonn et al., 2025). A PARSEME annotation is a set of token groups, each with a category, decided by tests such as "can a verbless noun phrase refer to the same event?" (Ramisch et al., 2018). Both schemes are complex in the sense that matters for annotation: the guideline is long, its decisions depend on each other, and a correct annotation needs both general understanding of the sentence and the scheme's conventions.

Procedures, skills and the registry. Decision orchestration compiles the guideline into a procedure Π: a set of *skills*, a *registry* that dispatches them, and numeric parameters θ. A skill has an input, an output schema (its *emit*) and an implementation, either code (a *rule node*) or a model call (a *decision node*). The registry runs *waves* in order; within a wave, a dispatcher maps fields of a skill's emit to the next skills, so control flow is a fixed function of typed decisions and no model can call an undeclared step. Figure 3 follows the UMR procedure through the sentence of Figure 2a.

(a) UMR: a sentence graph, its alignment, and document-level relations

And$_1$ *you*$_2$ *watch*$_3$ *him*$_4$ *pluck*$_5$ *a*$_6$ *few*$_7$ *pears*$_{8,9}$ (sentence 10 of a narrative)

```
(s10w / watch-01
  :ARG0 (s10p / person
     :refer-person 2nd :refer-number singular)
  :ARG1 (s10p2 / pluck-01
     :ARG0 (s10p3 / person
        :refer-person 3rd :refer-number singular)
     :ARG1 (s10p4 / pear :quant (s10a / a-few))
     :aspect activity)
  :aspect state)
```

```
alignment: s10w 3-3, s10p 2-2, s10p2 5-5,
  s10p3 4-4, s10p4 8-8, s10a 6-7
:temporal ((s8e :after s10w)
        (s10w :overlap s10p2))
:modal ((root :modal author)
      (author :full-affirmative s10w) ...)
:coref ((s8p :same-entity s10p)
      (s9p2 :same-entity s10p4))
```

(b) PARSEME: verbal multiword expressions

After that, he made$_6$ *a great* effort$_9$ *to be calm.*
{6, 9}: LVC.full (light verb + predicative noun, discontinuous)

So Harry had been brought$_5$ up$_6$ *by his dead mother's sister ...*
{5, 6}: VPC.full (the particle changes the verb's meaning)

我 已經 作出$_3$ 決定$_4$ ， 讓 他 發言 。
{3, 4}: LVC.full; {3}: VPC.semi (one token: 作 + complement 出)

Categories (EN 1.1): VID, LVC.full, LVC.cause, VPC.full, VPC.semi, IAV;
(ZH 1.2): VID, LVC.full, LVC.cause, VPC.semi, MVC.

Figure 2: The two annotation schemes, on training sentences. (a) UMR (Van Gysel et al., 2021): a rooted graph of concepts (PropBank senses, abstract concepts, words), roles and attributes per sentence, aligned to tokens, plus document-level temporal, modal and coreference relations that may point into earlier sentences; the guidelines run to over 300 pages. (b) PARSEME (Ramisch et al., 2018, 2020): sets of possibly discontinuous, possibly overlapping tokens with a category, decided by the guidelines' decision trees of linguistic tests; in Chinese a single token can be an expression.

`sent_type` decides what a piece of the sentence is (several coordinated clauses, one predicate, a noun phrase, a named entity or date, or a leaf); `predicate` chooses a head, its sense among the PropBank rolesets that code looked up, its aspect, modal strength and polarity, and the participant role of each word; every participant becomes a placeholder that goes back to `sent_type`; the rule node `coverage` sends back words nothing took. A rule wave merges nodes that stand for the same thing, and the `document` wave adds temporal, modal and coreference relations using the five previous sentences. The PARSEME procedure is the smallest instance: a rule node enumerates candidate token groups from the dependency tree of the input, a decision node judges each candidate, and the output is written by code.

Two decision modes. A decision node has two interchangeable implementations with the same emit. In a **fragment decision**, an LLM reads the skill's instructions (the part of the guideline the step needs), the piece and the sentence, may look a lemma up in PropBank, and answers with its emit through a schema: a small graph fragment with placeholders, a list of parts, a relation list. The schema is the format constraint: fields and enumerations are fixed by the registry, spans must lie inside the piece, every variable of an edge must be a node or a placeholder, and an answer that does not validate goes back to the model; no call ever writes a whole graph. In a **closed decision**, the skill's code enumerates what the answer could be (the words of a piece, the rolesets of a verb, the options of the guideline with a one-line description each) and asks closed questions: a choice among options, yes/no, or a graded score. A *decision model* returns a probability distribution over the options and no text; rules read the distributions against thresholds θ and build the emit (Figure 3). Closed decisions need no model that can write structured text and come with probabilities, so they can be served by decision services (TypeSafe, 2026; OpenAI, 2026; Microsoft, 2026), and every decision is logged with its alternatives.

Threshold calibration. A rule like "keep the candidate if $p(\text{yes}) \geq 0.5$" is written for some answerer, and another decision model spreads its probability mass differently. What matters is the decision the rule takes, so we fit θ to the task metric on the training sentences by coordinate descent: each threshold in turn is swept over $\{0.2, 0.3, \ldots, 0.8\}$ with the others fixed, a value is kept when it raises training F by more than 0.1. Only the rules are rerun; cached answers make calibration nearly free.

Cascade. Every closed answer carries a confidence $c = \max_o p(o)$. In the cascade, questions with $c < \tau$ are asked again, with the same state and options, of a small LLM (Claude Haiku 5.5) that must pick one of the options; its pick replaces the decision model's answer, and nothing else changes. τ is chosen on the training sentences.

Evaluation and reflection. A parse is scored with the task's official metric and with a fine-grained judge that compares predicted and gold units (nodes, concepts, senses, attributes, arguments, roles, names, reentrancies, temporal, modal and coreference relations; §4). Each predicted unit is attributed to the skill call that produced it, through the parse's trace, and each gold unit no call produced is marked as such. Reflection improves the procedure, never a model, one batch of five gold

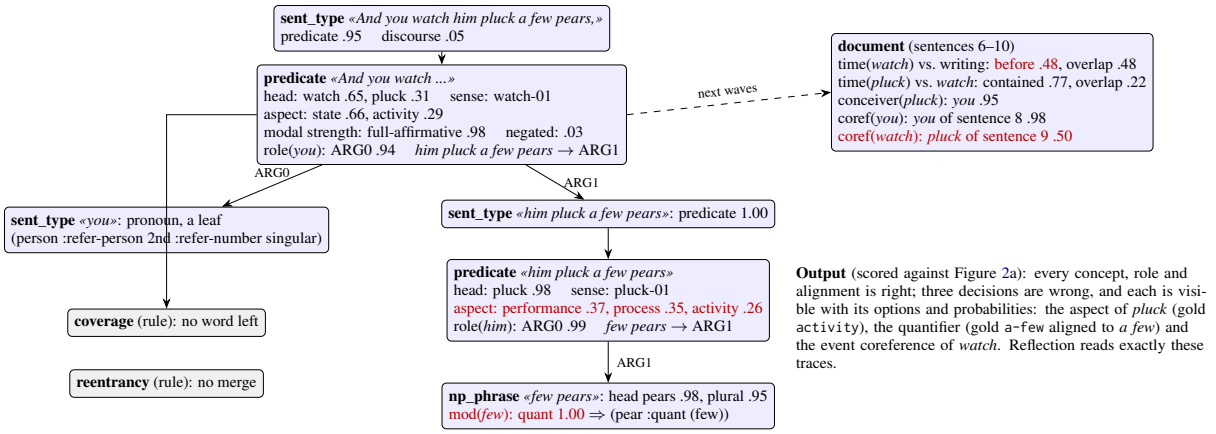


Figure 3: The closed-mode seed procedure on the sentence of Figure 2a (real trace, decisions by Jev). Blue: decision nodes, with the options code enumerated and the probabilities the decision model returned; grey: rule nodes. Red: decisions that differ from the gold.

sentences at a time (Algorithm 1, appendix). For UMR, the five sentences are consecutive sentences of one document, so that coreference and temporal relations within the batch are kept; documents are otherwise broken up. A round has four steps. (1) The batch is parsed with the current version and judged. (2) A reflector LLM reads the gold and predicted annotations, the judge's differences with their skills, every decision with its options and probabilities, and the editable files, lists *every* problem it finds (we set no limit) with its cause in the procedure, and proposes edits that modify, add or delete text in one file each. (3) Acceptance. In closed mode a parse costs seconds, so each edit is measured: it is applied to a copy, the batch and all earlier batches are re-parsed, and it is kept if the batch improves while the earlier batches do not fall by more than 0.5 F, or the earlier batches improve while the batch does not fall; rejected edits go back to the reflector once with their measured effect. In fragment mode a parse costs minutes, so an edit is accepted if it applies and the procedure still loads, and the next round tests it. (4) Compression: a file that has grown beyond 1.5 times its seed length is rewritten shorter by the reflector, merging rules and dropping redundant ones while keeping every option, question and convention; in closed mode the rewrite is measured like an edit. The result is a new frozen version, and its difference from the last is a readable diff. Edits that copy a stretch of test text (six Chinese characters or four English words) are refused.

4 Experiments

UMR data. We use the data of the UMR parsing shared task at DMR 2026 (Zeman and Štěpánek, 2026), whose training data is largely based on UMR 2.1 (Bonn et al., 2025). For testing we take the shared task's blind test documents for which we have the gold annotation: 13 Chinese documents (`zh-0021–0024`, `0027–0035`; 185 sentences) and the four English documents `en-0005–0008` (95 sentences; `0005` and `0007` are two annotations of the same text, kept as in the shared task). Documents are broken up: the Chinese test set is 20 chunks of five consecutive sentences, at least one per document (100 sentences), and the English one all 95 sentences in 20 chunks of at most five. Each chunk is parsed as a document of its own, and its gold keeps the document-level relations between nodes of the chunk. Input is tokenized as in the shared task. For training, reflection reads 10 batches of five consecutive sentences (50 sentences): in Chinese from 10 different training documents (`0001–0018`), in English from the native-annotation training documents that share no text with the test (the narrative `0004` and the news documents `0578–0579`; the training documents `0001` and `0003` are excluded, since their text is that of test documents `0006` and `0008`).

PARSEME data. PARSEME 2.0 (Savary et al., 2026; Scholivet et al., 2026) has no Chinese data and only a 16-sentence English trial set, so we use the latest editions with these languages: Chinese from edition 1.2 (Ramisch et al., 2020) and English from 1.1 (Ramisch et al., 2018), both verbal MWEs. Training: 10 batches of five training

sentences, three with a gold VMWE and two without. Test: 100 official test sentences, 50 with a VMWE and 50 without, sampled with a fixed seed so that 100 sentences contain enough expressions to score (中文 59, English 57 VMWEs). We re-score the shared-task systems' released predictions on exactly these sentences; on the full test files our scorer reproduces their published F (e.g., 71.86 for TRAVIS-mono, 33.27 for Milos).

Systems. Baseline: GPT-6.1-sol, a frontier model, annotates each test chunk directly, given only the full guideline (the UMR guidelines, 366k characters; the PARSEME 1.2 guidelines, 335k characters), the sentences (for PARSEME with the lemma, part-of-speech and dependency columns of the input) and the output format; no gold example. **Seed procedures** are written from the guidelines only (Appendix A): the instructions, options and tests restate the guideline, the UMR seeds follow the official file format's inventory of relations and attributes, and there is no example anywhere (every `example.md` of the fragment skills is empty). The fragment and closed seeds share the decomposition and differ in the decision nodes. **Fragment mode**: decisions by two small, cheap LLMs, GPT-6-luna and Claude Haiku 5.5 ($0.10 per million input tokens each), with low reasoning effort. **Closed mode**: decisions by three decision services, Jev 1.13 (TypeSafe, 2026), OpenAI's Decisions API with `gpt-6-luna-decisions` (OpenAI, 2026) and Microsoft-Decision-1 (Microsoft, 2026), all through the same interface; and the **cascade**, Jev with uncertain questions re-asked of Haiku 5.5. The **reflector** is Claude Sonnet 5.5 for every run. Each configuration runs ten rounds of reflection from its seed, and every frozen version $V_0, \ldots, V_{10}$ is tested. Within our budget and OpenRouter's limit of 20 requests a minute for OpenAI models, this was done for Haiku, Jev and Microsoft on both tasks and for GPT-6-luna on PARSEME; the OpenAI Decisions chains are reported as far as they ran (Limitations).

Metrics. UMR is scored with the shared task's official ju:mætʃ F1 (Zeman and Štěpánek, 2026): predicted nodes are mapped to gold nodes through their token alignment, and F1 is computed over the triples (concepts, relations, attributes) of mapped nodes, document-level relations included; we also report it without the document level ("no doc"). For diagnosis we use a fine-grained judge (Table 1): it maps nodes with AnCast (Sun and Xue, 2024), breaks the comparison into 25 checks in 11 modules, gives every unit a verdict and every check its own F1, and attributes each predicted unit to the skill call that produced it. PARSEME is scored with the official evaluation script: MWE-based F1 (exact token set and category), token-based F1, and F1 on expressions unseen in the training data. A PARSEME judge sorts every gold and predicted expression into six kinds: correct, wrong category (same tokens), partial (overlapping tokens), missed without a candidate (no rule proposed its tokens), missed after rejection (a candidate had them), and spurious (no overlap with any gold expression).

State of the art. For UMR we compare with the best shared-task submission (Zeman and Štěpánek, 2026), entered as a placeholder of 50 F in both languages until it is re-scored on our test chunks. For PARSEME, the best systems of the shared tasks (Chinese 1.2: TRAVIS-mono/-multi, open track; English 1.1: Milos, open track, and TRAPACC, closed track), re-scored on our sentences, and the best published English 1.1 result (Rohanian et al., 2019), quoted on the full test set.

5 Results

Table 2 gives the headline numbers and Figure 4 the test score of every version along the way. Appendix Table 7 lists every number.

The guideline as steps beats the guideline in context. With no gold data at all, the seed procedures already compete with a frontier model that reads the whole guideline. On Chinese UMR, every closed seed (Jev 33.9, Microsoft 35.0, OpenAI 32.1) is 8–11 points above GPT-6.1-sol (23.9), although each decision is taken by a model that cannot write text; on English UMR the seeds are within 1.5 points of it (29.0–29.8 against 30.2), the small LLMs as fragment deciders on par (Haiku 28.8). The one seed below the baseline, Haiku on Chinese UMR (20.9), fails in a way that the trace shows at once: it translated every Chinese concept into English (计划 → `plan-01`), which no guideline sentence forbids explicitly (event concepts 30.0 F in the judge, Table 3). On PARSEME, the fragment seeds start well above the frontier baseline in Chinese (GPT-6-luna 62.5, Haiku 59.5, against 50.0), while the closed seeds start low (22.5–37.8), for a reason we discuss below; in English every seed is below the frontier baseline (at most 62.3 against 75.6).

Module	Checks	What one unit compares (for matched gold/predicted nodes)	Unit kind
a. nodes	event found, entity found	does a gold event (entity) node have a predicted match; is a predicted one matched	node
	event concept, entity concept	is the concept the same	node pair
	granularity	is the node aligned to the same words (one node for several gold nodes or vice versa)	span
b. sense	event sense	same PropBank roleset number, once the concept is right	node pair
c. attributes	aspect, modal strength, polarity, refer, other	same value of the attribute; an attribute only one side has is missing or extra	attribute
d. arguments	core args of events / of abstract predicates; peripheral args of events / of entities	how far apart the argument sets of a matched head are (credit = share that agrees)	head
e. role labels	core, peripheral	same role label on a matched edge	edge
f. names	name tokens, wiki	same `:op` tokens of a matched name; same wiki title	entity
g. reentrancy	reentrancy	is an edge back to an already declared node also in gold	edge
h. temporal	temporal dependency	a child node's temporal parent(s) and relation(s)	child
i. modal	modal dependency	an event's conceiver (parent) and modal strength (label)	child
j. coreference	chains, subsets	is a mention linked to the same other mentions as in gold; subset relations	mention
k. format	document format	are the predicted document-level relations well formed (no gold needed)	relation

Table 1: The fine-grained UMR judge. Every unit gets a verdict: correct (credit 1), partial (0.5, or the check's own credit), wrong, missing or extra (0). Recall counts the gold side (correct, partial, wrong, missing), precision the predicted side (correct, partial, wrong, extra), and each check reports its F1. Modules a–f never see reentrant edges, so module g's errors do not move them. Each predicted unit is attributed to the skill call that wrote it.

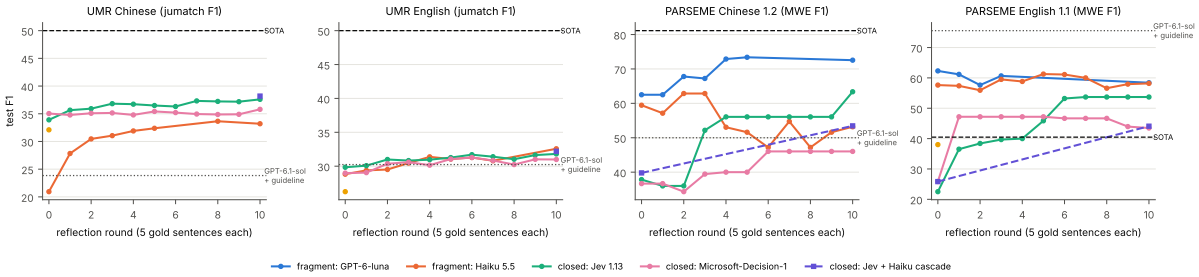


Figure 4: Test F1 of every frozen version $V_0, \ldots, V_{10}$; round r has read $5r$ gold sentences. Dotted: GPT-6.1-sol with the whole guideline; dashed: best shared-task system (UMR: placeholder).

Fifty gold sentences of reflection. Reflection improves twelve of the fourteen chains that finished. On Chinese UMR, Haiku climbs from 20.9 to 33.2 (best 33.6), the first round alone adding 6.9 points by writing the missing convention ("for Chinese the concept is the source word itself, with a sense number"), and Jev from 33.9 to 37.6, the best UMR result, 13.7 points above the frontier baseline. On English UMR the gains are smaller: Haiku 28.8 $\rightarrow$ 32.6, Jev 29.8 $\rightarrow$ 31.8, Microsoft 29.0 $\rightarrow$ 31.0, all above the baseline at V_{10}. On PARSEME, the measured closed chains gain most: Jev rises from 37.8 to 63.4 in Chinese and from 22.5 to 53.7 in English, Microsoft from 36.7 to 46.0 and from 25.9 to 43.5. The fragment chains on PARSEME, whose edits are not measured, move less predictably: GPT-6-luna rises in Chinese from 62.5 to 72.5 (best 73.4 at V_5), the best Chinese PARSEME result and 22.5 points above the frontier baseline, but falls in English from 62.3 to 58.4; Haiku reaches 62.9 in Chinese at V_3 and drifts back to 53.2 by V_{10} (§6).

Against the state of the art. On English PARSEME, every procedure after reflection is above the best system of the 1.1 shared task rescored on the same sentences (Milos, 40.5): Jev at 53.7, Haiku at 58.2, GPT-6-luna at 58.4, from 50 annotated sentences against the full training data. The English 1.1 systems predate pretrained language models, however, and the frontier model given the guideline is better still (75.6): in English, which these models know best, the guideline in context works, and decomposition with small deciders does not catch up within ten rounds. On Chinese PARSEME the 1.2 systems, trained on the full data with pretrained encoders, remain ahead (TRAVIS-multi 81.1 against GPT-6-luna's 72.5), although our best procedure is on par with the third open-track system (MTLB-STRUCT, 73.1) and above the best closed-track one (ERMI, 62.8). On UMR, our best procedures are 12–18 points below the placeholder for the best shared-task system.

5.1 Where fragment and closed decisions differ

Because every unit of an annotation is attributed to the step that made it, the two modes can be compared check by check (Table 3). On UMR the closed procedure wins on what code controls: *granularity* (which words a node covers; Chinese 92.9 against 85.8), *refer* (person and number of pro-

System	UMR (ju:mætʃ F1) Chinese	UMR (ju:mætʃ F1) English	PARSEME (MWE F1) Chinese	PARSEME (MWE F1) English
Best shared-task system	50.0†	50.0†	81.1	40.5
GPT-6.1-sol + full guideline	23.9	30.2	50.0	75.6
fragment: GPT-6-luna, seed V_0	–	–	62.5	62.3
V_{10}	–	–	72.5	58.4
best version	–	–	73.4 (V_5)	62.3 (V_0)
fragment: Haiku 5.5, seed V_0	20.9	28.8	59.5	57.6
V_{10}	33.2	32.6	53.2	58.2
best version	33.6 (V_8)	32.6 (V_{10})	62.9 (V_3)	61.3 (V_5)
closed: Jev 1.13, seed V_0	33.9	29.8	37.8	22.5
V_{10}	37.6	31.8	63.4	53.7
best version	37.6 (V_{10})	31.8 (V_{10})	63.4 (V_{10})	53.7 (V_{10})
closed: OpenAI Decisions, seed V_0	32.1	26.2	–	38.0
V_{10}	–	–	–	–
best version	32.1 (V_0)	26.2 (V_0)	–	38.0 (V_0)
closed: Microsoft-Decision-1, seed V_0	35.0	29.0	36.7	25.9
V_{10}	35.8	31.0	46.0	43.5
best version	35.8 (V_{10})	31.3 (V_6)	46.0 (V_{10})	47.2 (V_5)
Jev V_{10}, calibrated	–	0.0	61.4	49.5
Jev V_{10} + Haiku cascade	38.2	32.2	53.4	44.1

Table 2: Test F1. UMR: the shared task's ju:mætʃ metric on 100 Chinese and 95 English sentences; PARSEME: MWE-based F1 on 100 sentences per language. V_0 is the seed written from the guideline, V_{10} the procedure after ten rounds of reflection on five gold sentences each (50 sentences in all); "best version" is the best of $V_0..V_{10}$ on the test set and is shown only to describe the curves (Figure 4). †Placeholder until the best shared-task submission is re-scored on our chunks. PARSEME shared-task systems are trained on the full training data and re-scored on our sentences: TRAVIS-multi (Chinese 1.2) and Milos (English 1.1). Red dashes: runs not finished within our budget (see Limitations).

Check	Chinese Haiku (fragm.) V_0	Chinese Haiku (fragm.) V_{10}	Chinese Jev (closed) V_0	Chinese Jev (closed) V_{10}	English Haiku (fragm.) V_0	English Haiku (fragm.) V_{10}	English Jev (closed) V_0	English Jev (closed) V_{10}
event concept	30.0	77.3	73.5	77.9	66.7	72.5	49.4	55.6
entity concept	36.3	73.8	64.4	69.7	69.5	76.3	68.1	72.5
granularity	82.5	85.8	92.2	92.9	84.9	88.1	90.3	90.7
event sense	58.6	84.7	84.4	86.1	63.2	71.9	56.8	60.9
aspect	60.5	65.7	51.7	63.1	52.0	61.3	55.2	57.5
polarity	52.6	61.1	56.4	49.0	36.8	36.8	32.4	37.8
refer (person/number)	20.5	23.2	42.6	49.0	35.3	49.9	35.8	37.7
core args of events	22.2	45.6	37.8	35.7	27.5	29.5	23.0	24.9
peripheral args of events	12.3	25.3	15.4	20.9	9.0	17.6	16.8	14.0
core role labels	60.2	75.1	74.8	68.6	65.4	60.9	69.7	71.8
name tokens	50.3	52.0	46.3	52.2	61.6	67.6	45.4	46.9
reentrancy	9.7	24.4	18.3	20.4	11.4	12.6	9.8	10.5
temporal	4.5	11.1	16.1	18.5	6.7	14.5	13.7	16.8
modal	48.7	54.6	55.2	56.5	28.6	31.7	28.1	27.2
coreference	3.0	16.4	4.3	12.7	11.9	22.5	9.3	11.3
ju:mætʃ F1 (official)	20.9	33.2	33.9	37.6	28.8	32.6	29.8	31.8

Table 3: The fine-grained UMR judge (F1 per check, %) for fragment decisions by Haiku and closed decisions by Jev, seed and V_{10}. Selected checks; all 25 are in the released results.

Lang.	System	correct	wrong cat.	partial	missed, no cand.	missed, rejected	spurious	F
Chinese	GPT-6.1-sol	25	3	4	27	0	21	50.0
	GPT-6-luna V_0	23	12	1	6	17	17	62.5
	GPT-6-luna V_{10}	33	4	1	6	15	5	72.5
	Haiku 5.5 V_0	26	7	4	3	19	16	59.5
	Haiku 5.5 V_{10}	22	3	3	5	26	8	53.2
	Jev 1.13 V_0	7	28	12	8	4	80	37.8
	Jev 1.13 V_{10}	22	10	1	11	15	9	63.4
	Microsoft-Decision-1 V_0	9	24	11	8	7	78	36.7
	Microsoft-Decision-1 V_{10}	16	13	5	8	17	34	46.0
English	GPT-6.1-sol	46	5	4	2	0	22	75.6
	GPT-6-luna V_0	31	7	8	6	5	21	62.3
	GPT-6-luna V_{10}	27	6	8	7	9	17	58.4
	Haiku 5.5 V_0	30	4	8	7	8	19	57.6
	Haiku 5.5 V_{10}	25	7	9	6	10	14	58.2
	Jev 1.13 V_0	19	6	20	10	2	112	22.5
	Jev 1.13 V_{10}	23	6	4	16	8	19	53.7
	OpenAI Decisions V_0	11	12	9	18	7	29	38.0
	Microsoft-Decision-1 V_0	17	13	15	11	1	123	25.9
	Microsoft-Decision-1 V_{10}	10	10	5	18	14	11	43.5

Table 4: PARSEME error kinds on the test set (counts of gold or predicted expressions): correct; same tokens with the wrong category; overlapping tokens; gold missed because no rule proposed its tokens, or because a proposed candidate was rejected; spurious predictions. F is MWE-based, category-blind as in the official script.

nouns, filled from closed-class tables; 49.0 against 23.2) and temporal relations. The fragment procedure wins on what needs open-ended judgment: the arguments of events (Chinese core arguments 45.6 against 35.7), reentrancies (24.4 against 20.4) and coreference chains (16.4 against 12.7). ju:mætʃ maps nodes through their token alignment, so granularity errors cost twice, and Jev comes out ahead in Chinese (37.6 against 33.2).

In English the order flips (Haiku 32.6, Jev 31.8), and the judge says why: event concepts (72.5 against 55.6) and senses (71.9 against 60.9). The trace shows that most of the difference is not in a decision at all. Of Jev's 212 wrong English event concepts, 51 are inflected forms (*stolen*, *thought*, *fallen*, *trie*) that the seed's rule-based lemmatizer failed to reduce, against 20 for Haiku, which lemmatizes as it writes; Chinese, without inflection, has none. A decision model chooses among the options that code gives it, so the closed mode is exactly as good as its enumerator: when the right concept is not an option, no probability can rescue it. Such an error is cheap to fix once found (an irregular-verb table), and it is found by reading one column of the judge's output.

On PARSEME the PARSEME judge (Table 4) explains why the closed seeds start so low: they predict 121–175 expressions for 57–59 gold, of which 78–123 are spurious. Every closed question is a test from the guideline's decision trees, answered by a model that has never seen the scheme's sense of "idiomatic" or "light"; at a threshold of 0.5 most candidates pass. Reflection is where the closed procedure learns that sense: Jev's spurious predictions fall from 80 to 9 in Chinese and from 112 to 19 in English, through edits such as "有 is not a light verb", "no VID for single-character object nouns" or "drop IAV on a verb that already has a VPC", each measured on the training batches before it was kept. What the closed procedure cannot fix this way are expressions no rule proposes ("missed, no candidate": 11 in Chinese, 16 in English, against 3–7 for the fragment procedures, which may return expressions outside the candidate list).

5.2 Calibration, cascade and transfer

Table 5 shows the two closed-mode adjustments of §3. **Threshold calibration** helps the seeds of three of four PARSEME procedures (Jev English 22.5 → 27.1, Chinese 37.8 → 39.2, Microsoft English 25.9 → 29.9) but not the reflected ones: after ten rounds of measured reflection, the thresholds have already been tuned implicitly by the edits that were kept (a "lower idiom threshold" edit, for example), and the remaining training gain of calibration (Jev Chinese 88.2 → 89.6) does not carry over to the test set (63.4 → 61.4). With 50 sentences, thresholds fitted on the training data overfit quickly. **The cascade** routes questions Jev answers with confidence below τ to Haiku. On PARSEME it hurts: on the training sentences no τ improved Jev's V_{10} (English 76.7 without the cascade, 64.6–73.5 with it; Chinese 88.2 against 77.3–82.2), and at $\tau = 0.7$ the test score falls too (English 53.7 → 44.1, Chinese 63.4 → 53.5), while on the seeds it helps a little (37.8 → 39.8, 22.5 → 25.8). Once reflection has tuned PARSEME's rules to Jev's answers, replacing the hard ones with another model's breaks that fit. On UMR, whose closed questions are more open (which concept, which role), the cascade adds a little on the test set (Chinese 37.6 → 38.2, English 31.8 → 32.2, the best UMR results) for \$0.25–0.78 per hundred sentences, although the training sentences would not have selected it either (Chinese 37.5 without, 37.1 at $\tau = 0.6$). **Transfer.** The procedure Jev's chain learned works with another decision model, and in three of four cases better than the procedure that model's own chain learned. Answered by Microsoft-Decision-1, Jev's V_{10} scores 57.1 on Chinese PARSEME against 46.0 for Microsoft's own V_{10}, and 62.0, close to Jev's 63.4, once its thresholds are calibrated for Microsoft, which is the situation calibration is meant for; 36.6 against 35.8 on Chinese UMR; 44.2 against 43.5 on English PARSEME; only on English UMR is it lower (30.4 against 31.0). What reflection writes is mostly about the scheme, not about the model, and Jev's chain learned more of it than Microsoft's.

5.3 What reflection does

Table 6 summarizes the ten rounds. The reflector lists 8–11 problems per UMR batch and 2–4 per PARSEME batch, and proposes about one edit per problem. In closed mode, measurement keeps about half of the edits (54 of 109 for Jev on Chinese UMR; 13–16 of 36–46 on PARSEME); a typical rejected edit is right about the batch and neutral elsewhere ("money is quantity", gain 0.0), or invalid (an edit that broke a YAML file is caught by the loader and never measured). In fragment mode every edit that applies is kept, so the procedure grows faster (Chinese UMR: 36.8k → 50.8k characters despite 28 compressions) than in closed mode (70.5k → 73.7k, mostly code), where edits are short and replace text more often than they add it.

The kept edits read like an annotation manual's errata, and they are where the procedure's improvements come from. In Chinese UMR, Jev's kept edits concern discourse connectives ("limit `and-but`", "speech content is not discourse"), the abstract predicates of non-verbal clauses ("`exist-91` for X中有Y", "`have-place-91` only without a verb"), aspect ("add inceptive aspect") and the person/number of nouns; every one is a readable change to a named step, which an annotator can check and keep or revert. On the training batches themselves, the closed UMR procedures improve steadily: Jev's score on all earlier batches rises from 31.6 after round 1 to 38.9 after round 10, of which 3.7 points carry over to the unseen test documents.

6 Discussion

Diagnosability. A guideline in context fails silently: GPT-6.1-sol's Chinese UMR graphs are worse than our seeds', and nothing in its output says which of the 366k characters it misread. Each error of a procedure has an address, a step, its input, its options and their probabilities, and that changes how errors are found and fixed. The Haiku seed's English concepts for Chinese words, Jev's unlemmatized English verbs and the closed PARSEME seeds' flood of spurious light-verb constructions (§5.1) were each found from one column of the judge's output and fixed by one edit to one file: by the reflector for the first and the third, and the second is a one-line change to the code that lists the options. The same address makes reflection possible at all: the reflector reads the decisions behind each difference, not just the difference, and its edits are diffs that an annotator can review, keep or revert. We see this as the main practical argument for the approach early in a project, when people and models correct each other.

Decision models are cheap and promising. Decision models never write text: they return prob-

Procedure	UMR zh raw	UMR zh cal.	UMR en raw	UMR en cal.	PARSEME zh raw	PARSEME zh cal.	PARSEME en raw	PARSEME en cal.
Jev 1.13 V_0	33.9	–	29.8	–	37.8	39.2	22.5	27.1
Jev 1.13 V_{10}	37.6	–	31.8	0.0	63.4	61.4	53.7	49.5
Microsoft-Decision-1 V_0	35.0	–	29.0	–	36.7	35.0	25.9	29.9
Microsoft-Decision-1 V_{10}	35.8	–	31.0	–	46.0	46.0	43.5	43.5
Jev V_{10} answered by Microsoft	36.6	–	30.4	–	57.1	62.0	44.2	42.3
Jev V_0 / + cascade	33.9	–	29.8	–	37.8	39.8	22.5	25.8
Jev V_{10} / + cascade	37.6	38.2	31.8	32.2	63.4	53.4	53.7	44.1

Table 5: Closed procedures with their thresholds as written (raw) and calibrated on the 50 training sentences (cal.); Jev's V_{10} answered by Microsoft-Decision-1; and the cascade to Haiku 5.5 at $\tau = 0.7$.

Task	Decisions	problems	edits	kept	compr.	size V_0	size V_{10}
UMR zh	Haiku 5.5	108	57	55	28	36.8k	50.8k
	Jev 1.13	108	109	54	0	70.5k	73.7k
	Microsoft-Decision-1	114	106	54	0	70.5k	74.8k
UMR en	Haiku 5.5	100	59	58	17	36.8k	51.5k
	Jev 1.13	87	89	35	0	70.5k	72.6k
	Microsoft-Decision-1	85	71	35	0	70.5k	72.7k
PARSEME zh	GPT-6-luna	26	20	18	12	4.5k	8.0k
	Haiku 5.5	29	18	18	9	4.5k	7.5k
	Jev 1.13	37	38	16	2	8.0k	9.6k
	Microsoft-Decision-1	38	46	13	0	8.0k	9.2k
PARSEME en	GPT-6-luna	24	18	18	8	4.7k	8.1k
	Haiku 5.5	26	21	21	9	4.7k	8.1k
	Jev 1.13	33	36	13	0	8.1k	9.7k
	Microsoft-Decision-1	38	31	8	0	8.1k	9.2k

Table 6: Ten rounds of reflection: problems the reflector listed, edits tried (with closed-mode retries), edits kept, compressions, and the size of the editable procedure (characters; closed mode includes code).

abilities over options that code has listed. This makes them the cheapest deciders we tried. Annotating the 100 Chinese UMR test sentences costs $0.28 with Jev and $0.19 with Microsoft-Decision-1, against $1.88 with Haiku in fragment mode and $7.09 for GPT-6.1-sol with the guideline; the 100 PARSEME sentences cost about one cent with either service. A closed parse is fast enough that every edit of reflection can be measured, which is what keeps closed-mode reflection from drifting. And the closed procedure is, as of now, the more accurate one on Chinese UMR (Jev 37.6) and close on English (31.8 against 32.6). Its ceiling is set by the code around it: a decision model cannot pick an option that is not offered (the lemmatizer, §5.1; PARSEME expressions no rule proposes) and cannot write structure. Better enumerators, which are ordinary code, raise that ceiling for every decision model at once, and the transfer result (§5.2) suggests that a procedure learned with one decision model can be handed to another.

What reflection can and cannot do. With five gold sentences a round, reflection writes the conventions a guideline leaves implicit (Chinese concepts stay Chinese; 有 is not a light verb; 但 is `but-91`) and fixes procedural errors (a missing candidate kind, a YAML convention). It does so from very little data: the largest gains come in the first rounds (Haiku Chinese UMR +6.9 in round 1; Jev Chinese PARSEME +20.1 in rounds 3–4). Three limits are visible. (1) *Unmeasured edits drift.* In fragment mode we accept every edit that applies, and Haiku's Chinese PARSEME procedure lost almost ten points between rounds 3 and 10 although each edit was locally sensible; measured acceptance gives the closed chains curves that rarely go down. Measuring fragment edits costs a reparse per edit, which a cheap fragment decider would make affordable. (2) *The training batches are tiny.* Closed edits are accepted on changes of a few points on five sentences, and the gap between training and test grows over the rounds (Jev Chinese PARSEME: 88.2 on training batches, 63.4 on test); threshold calibration on the same 50 sentences overfits (§5.2). More gold makes both checks stronger, and the procedure keeps improving as gold arrives. (3) *Reflection edits what exists.* It adds rules, options and questions, but in ten rounds it rarely changed the decomposition itself (which step decides what); in English PARSEME, where the frontier model with the guideline reaches 75.6, the procedures' gains flatten well below it.

When to decompose. The frontier model in context is strongest where the scheme is close to what such models already know and the output is a short list, English verbal MWEs; decomposition is strongest where the output is structure and the conventions are particular, Chinese UMR. For a new scheme or language early in a project, the procedure costs a few dollars to run, improves from the first batches of gold, and leaves a trace in which every error can be found.

7 Conclusion

We annotated two complex schemes step by step along their guidelines. Compiled into a procedure of rule nodes and small decisions, a guideline that is too long to follow in context becomes something small and cheap models can follow: seeds written from the guideline alone, with no example, match or beat a frontier model reading the whole guideline on three of four task–language pairs, 8–11 points ahead of it on Chinese UMR. Every error has an address, so it can be traced to the step that made it, and a reflector can fix the step: from fifty gold sentences, five at a time, the procedures improved in twelve of fourteen runs, by up to 31 points, and on English PARSEME every reflected procedure is above the best shared-task system on the same sentences. Decision models that only return probabilities, at a few cents per hundred sentences, take the best UMR result. The procedure keeps a readable history of what it learned, and the more gold arrives, the more it has to learn from. We think this makes decision orchestration a natural first step for a new annotation project: start from the guideline, annotate with cheap models, and let the procedure improve as the first gold comes in.

Limitations

Scale. Test sets are 95–100 sentences per task and language and training is 50 sentences; each chain was run once, with one document order. With 57–59 gold expressions per PARSEME test set, one expression is worth about one point of F, so differences of a few points there should not be overread; we did not repeat chains with other document orders to measure the variance of reflection. **Unfinished runs.** OpenRouter limits each OpenAI model to 20 requests a minute for our account, and our API budget was fixed. GPT-6-luna as a UMR fragment decider needs about 3,000 calls per test pass, and the OpenAI Decisions chains measure every edit; we therefore report OpenAI Decisions on UMR only as seeds, did not run GPT-6-luna on UMR, and finished fewer rounds for the OpenAI chains on PARSEME (red dashes in the tables). **Data editions.** PARSEME 2.0 has no Chinese data and only an English trial set, so we use editions 1.2 (Chinese) and 1.1 (English); the English 1.1 systems predate pretrained models. There is no UMR 3.0 release; we use the 2026 shared-task data, which is based on UMR 2.1, and the shared task's best result is a placeholder until it is re-scored on our test chunks. **Seeds.** The seeds were written by the authors (with an LLM's help) from the guidelines; their quality affects every result and we have not measured the effort they take. Closed and fragment seeds share the decomposition, so the comparison between modes is not a comparison of decompositions. **Reflection.** Fragment-mode edits are accepted unmeasured and drift; the same reflector (Claude Sonnet 5.5) was used throughout. **Models.** All models are commercial services whose behavior may change; we release the predictions, the procedures at every version, and the cached answers that their terms permit.

Ethics Statement

We use the UMR shared-task data and PARSEME 1.1 and 1.2 under their licenses (PARSEME 1.2 Chinese: CC BY-NC-SA 4.0) and collect no data from people. The method aims to make the first annotations of a new scheme cheaper and easier to check; it could reduce demand for annotation work, and we see it rather as a way to direct annotators to the decisions that need them. Automatic annotations should be checked before they are used as gold. The runs of this version cost about $59 in logged API charges (decision services $13, LLM calls $46), including runs that were stopped; reflection used a subscription. Generative AI assisted the implementation, the running of the experiments and the editing of this paper; the authors are responsible for all content.

References

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *The Fourteenth International Conference on Learning Representations*.

Xuefeng Bai, Yulong Chen, and Yue Zhang. 2022. Graph pre-training for AMR parsing and generation. *arXiv preprint arXiv:2203.07836*.

Michele Bevilacqua, Rexhina Blloshmi, and Roberto Navigli. 2021. One SPRING to rule them both: Symmetric AMR semantic parsing and generation without a complex pipeline. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(14):12564–12573.

Julia Bonn, Claire Bonial, Matt Buchholz, Hsiao-Jung Cheng, Alvin Chen, Ching-wen Chen, Andrew Cowell, William Croft, Lukas Denk, Ahmed Elsayed, Eva

Fučíková, Federica Gamba, Carlos Gomez, Jan Hajič, Eva Hajičová, Jiří Havelka, Loden Havenmeier, Ath Kilgore, Veronika Kolářová, and 40 others. 2025. Uniform meaning representation 2.0.

Eric Brill. 1992. A simple rule-based part of speech tagger. In *Proceedings of the Third Conference on Applied Natural Language Processing*, pages 152–155. Association for Computational Linguistics.

Eric Brill. 1995. Transformation-based error-driven learning and natural language processing: A case study in part-of-speech tagging. *Computational Linguistics*, 21(4):543–565.

Jayeol Chun and Nianwen Xue. 2024. A pipeline approach for parsing documents into uniform meaning representation graphs. In *Proceedings of the Fifth International Workshop on Designing Meaning Representations @ LREC-COLING 2024*.

William W. Cohen. 1995. Fast effective rule induction. In *Proceedings of the Twelfth International Conference on Machine Learning*, pages 115–123.

Allyson Ettinger, Jena D. Hwang, Valentina Pyatkin, Chandra Bhagavatula, and Yejin Choi. 2023. "you are an expert linguistic annotator": Limits of llms as analyzers of abstract meaning representation. *arXiv preprint arXiv:2310.17793*.

Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. 2023. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30):e2305016120.

Irving John Good. 1966. Speculations concerning the first ultraintelligent machine. In *Advances in Computers*, volume 6, pages 31–88. Academic Press.

Keno Harada and 1 others. 2025. When instructions multiply: Measuring and estimating LLM capabilities of multiple instructions following. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics.

Xingwei He, Zhenghao Lin, Yeyun Gong, A-Long Jin, Hang Zhang, Chen Lin, Jian Jiao, Siu Ming Yiu, Nan Duan, and Weizhu Chen. 2024. AnnoLLM: Making large language models to be better crowdsourced annotators. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pages 165–190, Mexico City, Mexico. Association for Computational Linguistics.

Shengran Hu, Cong Lu, and Jeff Clune. 2025. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations*.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations*.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, and 1 others. 2024. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*.

Kon Woo Kim, Jin-Dong Kim, and Akiko Aizawa. 2026. Refining and reusing annotation guidelines for LLM annotation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 37951–37964, San Diego, California, United States. Association for Computational Linguistics.

Douglas B. Lenat. 1983. EURISKO: A program that learns new heuristics and domain concepts. the nature of heuristics III: Program design and results. *Artificial Intelligence*, 21(1–2):61–98.

Minzhi Li, Taiwei Shi, Caleb Ziems, Min-Yen Kan, Nancy Chen, Zhengyuan Liu, and Diyi Yang. 2023. CoAnnotating: Uncertainty-guided work allocation between human and large language models for data annotation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 1487–1505, Singapore. Association for Computational Linguistics.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Aman Madaan, Niket Tandon, Prakhar Gupta, and 1 others. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*.

Emma Markle, Javier Gutierrez Bach, and Shira Wein. 2025. Setup: Sentence-level english-to-uniform meaning representation parser. *arXiv preprint arXiv:2512.07068*.

Microsoft. 2026. Use Microsoft-Decision-1 in Azure AI Foundry. `https://learn.microsoft.com/en-us/azure/foundry/foundry-models/how-to/use-foundry-models-microsoft-decision`. Accessed October 2026.

Tom M. Mitchell, Paul E. Utgoff, and Ranan Banerji. 1983. Learning by experimentation: Acquiring and refining problem-solving heuristics. In Ryszard S. Michalski, Jaime G. Carbonell, and Tom M. Mitchell, editors, *Machine Learning: An Artificial Intelligence Approach*. Tioga.

Grace Ngai and Radu Florian. 2001. Transformation-based learning in the fast lane. In *Proceedings of the Second Meeting of the North American Chapter of the Association for Computational Linguistics*.

Alexander Novikov and 1 others. 2025. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*.

OpenAI. 2026. Decisions API guide. https://developers.openai.com/api/docs/guides/decisions. Accessed October 2026.

Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. 2024. Optimizing instructions and demonstrations for multi-stage language model programs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9340–9366, Miami, Florida, USA. Association for Computational Linguistics.

Chaoxu Pang, Yixuan Cao, Qiang Ding, and Ping Luo. 2023. Guideline learning for in-context information extraction. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 15372–15389, Singapore. Association for Computational Linguistics.

Nicholas Pangakis, Samuel Wolken, and Neil Fasching. 2023. Automated annotation with generative AI requires validation. *arXiv preprint arXiv:2306.00176*.

Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 2023. Automatic prompt optimization with "gradient descent" and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*.

Carlos Ramisch, Silvio Ricardo Cordeiro, Agata Savary, Veronika Vincze, Verginica Barbu Mititelu, Archna Bhatia, and 1 others. 2018. Edition 1.1 of the PARSEME shared task on automatic identification of verbal multiword expressions. In *Proceedings of the Joint Workshop on Linguistic Annotation, Multiword Expressions and Constructions (LAW-MWE-CxG-2018)*, pages 222–240, Santa Fe, New Mexico, USA. Association for Computational Linguistics.

Carlos Ramisch, Agata Savary, Bruno Guillaume, Jakub Waszczuk, Marie Candito, Ashwini Vaidya, Verginica Barbu Mititelu, Archna Bhatia, Uxoa Iñurrieta, Voula Giouli, Tunga Güngör, Menghan Jiang, Timm Lichte, Chaya Liebeskind, Johanna Monti, Renata Ramisch, Sara Stymne, Abigail Walsh, and Hongzhi Xu. 2020. Edition 1.2 of the PARSEME shared task on semi-supervised identification of verbal multiword expressions. In *Proceedings of the Joint Workshop on Multiword Expressions and Electronic Lexicons*, pages 107–118. Association for Computational Linguistics.

Lance Ramshaw and Mitch Marcus. 1995. Text chunking using transformation-based learning. In *Third Workshop on Very Large Corpora*.

Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. 2017. Snorkel: Rapid training data creation with weak supervision. *Proceedings of the VLDB Endowment*, 11(3):269–282.

Omid Rohanian, Shiva Taslimipoor, Samaneh Kouchaki, Le An Ha, and Ruslan Mitkov. 2019. Bridging the gap: Attending to discontinuity in identification of multiword expressions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2692–2698, Minneapolis, Minnesota. Association for Computational Linguistics.

Oscar Sainz, Iker García-Ferrero, Rodrigo Agerri, Oier Lopez de Lacalle, German Rigau, and Eneko Agirre. 2024. GoLLIE: Annotation guidelines improve zero-shot information-extraction. In *The Twelfth International Conference on Learning Representations*.

Agata Savary, Manon Scholivet, Carlos Ramisch, Takuya Nakamura, Eric Bilinski, Sara Stymne, and 1 others. 2026. PARSEME 2.0 multilingual corpus of multiword expressions. In *Proceedings of the Fifteenth Language Resources and Evaluation Conference*, pages 4819–4834.

Jürgen Schmidhuber. 2007. Gödel machines: Fully self-referential optimal universal self-improvers. In *Artificial General Intelligence*, pages 199–226. Springer.

Manon Scholivet, Agata Savary, Carlos Ramisch, Eric Bilinski, Takuya Nakamura, Maria Mitrofan, and Vasile Pais. 2026. Edition 2.0 of the PARSEME shared task on multilingual identification and paraphrasing of multiword expressions. In *Proceedings of the 22nd Workshop on Multiword Expressions (MWE 2026)*, pages 254–275. Association for Computational Linguistics.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*.

Haibo Sun and Nianwen Xue. 2024. Anchor and broadcast: An efficient concept alignment approach for evaluation of semantic graphs. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 1052–1062, Torino, Italia. ELRA and ICCL.

Haibo Sun, Nianwen Xue, Jin Zhao, Liulu Yue, Yao Sun, Keer Xu, and Jiawei Wu. 2024. Chinese UMR annotation: Can LLMs help? In *Proceedings of the Fifth International Workshop on Designing Meaning Representations @ LREC-COLING 2024*, pages 131–139.

Zhen Tan, Dawei Li, Song Wang, Alimohammad Beigi, Bohan Jiang, Amrita Bhattacharjee, Mansooreh Karami, Jundong Li, Lu Cheng, and Huan Liu. 2024. Large language models for data annotation and synthesis: A survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 930–957, Miami, Florida, USA. Association for Computational Linguistics.

Petter Törnberg. 2023. ChatGPT-4 outperforms experts and crowd workers in annotating political Twitter messages with zero-shot learning. *arXiv preprint arXiv:2304.06588*.

TypeSafe. 2026. Jev API reference. `https://docs.typesafe.ai/api`. Accessed October 2026.

Jens E. L. Van Gysel, Meagan Vigus, Jayeol Chun, Kenneth Lai, Sarah Moeller, Jiarui Yao, Tim O'Gorman, Andrew Cowell, William Croft, Chu-Ren Huang, Jan Hajič, James H. Martin, Stephan Oepen, Martha Palmer, James Pustejovsky, Rosa Vallejos, and Nianwen Xue. 2021. Designing a uniform meaning representation for natural language processing. *KI - Künstliche Intelligenz*, 35(0):343–360.

Shuohang Wang, Yang Liu, Yichong Xu, Chenguang Zhu, and Michael Zeng. 2021. Want to reduce labeling cost? GPT-3 can help. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 4195–4205, Punta Cana, Dominican Republic. Association for Computational Linguistics.

Xingyao Wang, Sha Li, and Heng Ji. 2023. Code4Struct: Code generation for few-shot event structure prediction. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3640–3663, Toronto, Canada. Association for Computational Linguistics.

Xinru Wang, Hannah Kim, Sajjadur Rahman, Kushan Mitra, and Zhengjie Miao. 2024. Human-LLM collaborative annotation through effective verification of LLM labels. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*, pages 1–21, Honolulu, HI, USA. ACM.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. In *International Conference on Learning Representations*.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. Optimizing generative AI by backpropagating language model feedback. *Nature*.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. 2024. Self-taught optimizer (STOP): Recursively self-improving code generation. In *First Conference on Language Modeling*.

Daniel Zeman and Jan Štěpánek. 2026. UMR parsing shared task at DMR 2026. `https://ufal.mff.cuni.cz/umr-parsing`. Data, evaluation script and results; accessed October 2026.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. 2026a. Darwin Gödel machine: Open-ended evolution of self-improving agents. In *International Conference on Learning Representations*. ArXiv:2505.22954.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2025. AFlow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations*.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026b. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations*.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17):19632–19642.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. Large language models are human-level prompt engineers. In *International Conference on Learning Representations*.

Zhaocheng Zhu, Yuan Xue, Xinyun Chen, Denny Zhou, Jian Tang, Dale Schuurmans, and Hanjun Dai. 2023. Large language models can learn rules. *arXiv preprint arXiv:2310.07064*.

Caleb Ziems, William Held, Omar Shaikh, Jiaao Chen, Zhehao Zhang, and Diyi Yang. 2024. Can large language models transform computational social science? *Computational Linguistics*, 50(1):237–291.

A The seed procedures

All seeds were written from the guidelines before any training sentence was read, and contain no example: no annotated sentence, no example word in an option description, and every fragment skill's `reference/example.md` is empty. They are in `experiment/orchestration-v2/seeds/`.

UMR. Both modes share the registry: the sentence wave (`sent_type`, `discourse`, `predicate`, `np_phrase`, `special_entity`, and the rule nodes `coverage` and `antecedent`), the rule wave `reentrancy`, and the `document` wave. Each decision skill restates one part of the guidelines: `sent_type` the criteria for events (Part 3-1-1: processes in any packaging, states and entities only when predicated) and the kinds of pieces; `discourse` the lattice of discourse relations (3-1-6); `predicate` light verbs (3-1-3-5), the abstract predicates of non-verbal clauses and their roles (Tables 4 and 8), word senses (3-1-4), participant and

non-participant roles (3-2), the aspect decision procedure (3-3-1: event nominals, then habitual, state, activity, endeavor, performance), polarity (3-3-3) and modal strength (4-3-1); `np_phrase` inverse roles for relative clauses, participant nominalizations and relational nouns (3-2-1-3), modifiers (3-2-2-2), quantity and unit (2-2-5) and person and number (3-3-5); `special_entity` the named-entity types of Table 5, dates and quantities; `document` the temporal superstructure and relations (4-2), the modal dependency (4-3) and coreference (4-1). In fragment mode each is a `skill.md` of 2.5–6.5k characters (35k in all) with the output schema; in closed mode each is a Python file that builds the questions and rules, with the model-facing text in a `prompts.yaml`. Function-word, pronoun and numeral tables are closed-class grammar, not annotation decisions.

PARSEME. A rule node proposes candidates from the input's dependency tree: in English a verb with its particle (`compound:prt`), with the head noun of its object or oblique, or with the adposition of its oblique; in Chinese a multi-character verb token, a verb with the verb or adjective right after it that depends on it, a verb with its object, and four-character tokens. In closed mode, each candidate is asked the yes/no tests of its kind from the guidelines' decision trees (LVC.0–LVC.5, VID.1–VID.5, VPC.1–VPC.3, IAV, and the Chinese MVC tree), and rules walk the tree on the answers with one threshold. In fragment mode the LLM reads a `decide.md` that restates the categories and tests, sees the candidates, and returns the expressions of the sentence, which may include expressions the rules did not propose.

B Reflection

The reflector's instructions (abridged): the procedure follows a written guideline, code cuts, enumerates and assembles, decision nodes judge; list every problem you can find, however small, with its cause in the procedure; propose edits (`replace`, `insert_after`, `delete`, each an exact anchor in one file) that fix them on these sentences and on unseen text; prefer general rules and better option descriptions to special cases; keep every file valid; when you add a rule, delete or merge what it makes redundant. The closed-mode variant lists the useful kinds of edits (the right answer among the options with a rule branch; sharper option descriptions; questions added, split or removed; thresholds; conventions written into rules; code that cuts or assembles wrongly), the fragment-mode variant its own (conventions, misleading instructions removed, explicit decision procedures, output format). The full prompts are in `code/reflect.py`.

Algorithm 1 Reflection on five gold sentences per round.

Require: seed V_0, training batches $B_1, \ldots, B_{10}$, mode $\in$ {closed, fragment}
for $r = 1, \ldots, 10$ **do**
  $V \leftarrow V_{r-1}$; parse B_r with V; judge; attribute each difference to a step
  $P, E \leftarrow$ REFLECTOR(gold, prediction, differences, decisions, files of V, history)
  for each edit $e \in E$ **do**
    if e applies, the procedure loads, and e copies no test text **then**
      if mode = closed **then**
        $\Delta_b, \Delta_g \leftarrow$ change of F on B_r and on $B_1..B_{r-1}$ with e
        keep e iff $(\Delta_b > 0 \wedge \Delta_g \geq -0.5) \vee (\Delta_g > 0 \wedge \Delta_b \geq 0)$
      else keep e
      end if
    end if
  end for
  closed: send rejected edits back once with their measured effect; try the rewrites
  for each file f longer than $1.5\times$ its seed **do**
    rewrite f shorter (reflector); closed: keep it iff no set falls by more than 0.5
  end for
  $V_r \leftarrow V$ (frozen; tested on the test set)
end for

C Data details

UMR. Splits are frozen by `code/splits.py` (seed 20261012) in `experiment/orchestration-v2/data/`. Chinese training: one batch of five consecutive sentences from each of ten training documents among `chinese_umr-0001`–`0018`; Chinese test: 20 chunks of five consecutive sentences from the 13 blind-test documents with gold (`0021`–`0024`, `0027`–`0035`), at least one chunk per document. English training: five, three and two batches from `english_umr-0004`, `0578` and `0579`; English test: all 95 sentences of `english_umr-0005`–`0008` in 20 chunks. Input tokens are the gold `Words` lines, as in the shared task. Gold is rewritten in the strict format the official scorer reads (one graph per sentence, alignments restricted to graph variables, document-level relations restricted to the chunk). Scoring: `compare_umr.pl -tsv`, with and without `-no-document-level`.

PARSEME. Chinese: PARSEME 1.2 `ZH`; English: PARSEME 1.1 `EN`. Training batches: three sentences with at least one VMWE and two without, ten batches. Test: 50 sentences with and 50 without a VMWE, drawn from the official test file. Unseen F uses a training file reduced to the sentences with a VMWE, which gives the same seen/unseen split as the full file and runs in seconds. The shared-task systems' released predictions are re-scored on the same 100 sentences with the official `evaluate.py`.

Guidelines. The UMR guidelines (`umr4nlp/umr-guidelines`, `guidelines.md`, 366k characters) and the PARSEME 1.2 guidelines (335k characters) are given to the baseline in full and were the only source of the seeds.

D All test scores

E Costs

F Earlier experiments: fragment decisions by Claude Sonnet 5.5

Before the experiments of the main text, we ran the fragment-mode UMR procedure with a stronger parser, Claude Sonnet 5.5, on Chinese UMR, with an earlier hand-written seed that contained worked examples, an earlier test protocol (whole documents; AnCast++ instead of the shared task's metric) and unmeasured reflection by Claude Opus 5.5 on one whole training document per round. These numbers are not comparable with the main text (different seed, test documents, metric and gold exposure) and are reported for completeness.

One training document raised the first test set from 45.3 to 50.8 F and a second to 52.6, with temporal relations improving most; third and fourth documents left the test scores flat. Unmeasured reflection accepted 440 of 441 edits and the skills grew from 41k to 156k characters in ten rounds, which motivated the length compression of the main text.

Task	Decisions	V_0	V_1	V_2	V_3	V_4	V_5	V_6	V_7	V_8	V_9	V_{10}
UMR zh	Haiku 5.5	20.9	27.8	30.5	31.1	31.9	32.4			33.6		33.2
	Jev 1.13	33.9	35.6	35.9	36.8	36.7	36.5	36.3	37.3	37.2	37.2	37.6
	OpenAI Decisions	32.1										
	Microsoft-Decision-1	35.0	34.8	35.1	35.1	34.8	35.4	35.2	35.0	34.9	34.9	35.8
	Jev + cascade											38.2
UMR en	Haiku 5.5	28.8	29.4	29.5	30.5	31.4	31.1	31.3	30.8			32.6
	Jev 1.13	29.8	30.1	31.0	30.8	31.0	31.3	31.7	31.4	31.0	31.6	31.8
	OpenAI Decisions	26.2										
	Microsoft-Decision-1	29.0	29.1	30.4	30.6	30.1	31.1	31.3	30.8	30.2	31.0	31.0
	Jev + cascade											32.2
PARSEME zh	GPT-6-luna	62.5	62.5	67.8	67.2	72.9	73.4					72.5
	Haiku 5.5	59.5	57.1	62.9	62.9	53.1	51.6	47.2	54.7	47.2	51.6	53.2
	Jev 1.13	37.8	36.0	36.0	52.2	56.1	56.1	56.1	56.1	56.1	56.1	63.4
	Microsoft-Decision-1	36.7	36.7	34.3	39.4	40.0	40.0	46.0	46.0	46.0	46.0	46.0
	Jev + cascade	39.8										53.4
PARSEME en	GPT-6-luna	62.3	61.2	57.6	60.7							58.4
	Haiku 5.5	57.6	57.4	55.9	59.5	58.8	61.3	61.1	60.0	56.6	57.9	58.2
	Jev 1.13	22.5	36.5	38.4	39.7	40.0	45.9	53.2	53.7	53.7	53.7	53.7
	OpenAI Decisions	38.0										
	Microsoft-Decision-1	25.9	47.2	47.2	47.2	47.2	47.2	46.7	46.7	46.7	44.0	43.5
	Jev + cascade	25.8										44.1

Table 7: Test F1 of every frozen version (UMR: juːmætʃ; PARSEME: MWE-based). Blank: not tested (budget).

	UMR		PARSEME	
Decisions	zh	en	zh	en
GPT-6.1-sol + guideline	7.09	5.09	4.05	4.02
Haiku 5.5 (fragment)	1.88	0.83	0.03	0.02
GPT-6-luna (fragment)	–	–	0.01	0.01
Jev 1.13 (closed)	0.28	0.10	0.01	0.01
Microsoft-Decision-1 (closed)	0.19	0.07	0.01	0.01

Table 8: API cost (USD) of annotating the test set (95–100 sentences) with the seed procedure, uncached, as billed through OpenRouter. Reflection used Claude Sonnet 5.5 through a subscription; its list-price equivalent is $0.4–3.2 per ten-round chain.

System	Refl. docs (sent.)	T1: 0004–05 F/ Comp.	T2: 0005–07 F/ Comp.
Single call, Sonnet 5.5	0	32.7 / 27.7	29.4 / 24.9
R_0 (seed), Sonnet 5.5	0	45.3 / 44.0	–
R_{13}	1 (51)	50.8 / 49.4	–
R_{23}	2 (111)	52.6 / 51.2	–
R_{32}	3 (142)	52.6 / 51.1	55.2 / 52.1
R_{42}	4 (201)	n/a	55.6 / 52.2

Table 9: Earlier Chinese UMR runs with Sonnet 5.5 fragment decisions (AnCast++ sentence-graph F and comprehensive score, macro-averaged over documents). R_{xy}: the version after round y on training document x. "n/a": 0004 was a training document by then.

Doc	seed	iter 1	iter 2	iter 3	after
0001	40.3	40.3	59.6	66.9	70.9
0002	50.6	58.9	66.3	70.4	73.4
0003	49.7	57.5	69.5	–	–
0004	39.6	49.7	57.9	–	–

Table 10: The same runs: sentence F on each training document over the rounds of reflection on it. Training documents gain far more than test documents, as in the main text.